Python File Handling

Dilini Chamika Silva

Overview

File handling in Python allows you to create, read, write, and modify files stored on disk. Python provides built-in functions and modules that make file operations simple and efficient.

1. What is File Handling?

File handling refers to working with files, such as:

- Creating files
- Opening files
- Reading data from files
- Writing data to files
- Appending data
- Closing files

Python mainly uses the built-in `open()` function for file handling

2. Opening a File

Syntax

```
file_object = open("filename", "mode")
```

Common File Modes

Mode	Description
r	Read (default) – error if file does not exist
w	Write – creates new file or overwrites existing file
a	Append – adds data to the end of the file
x	Create – error if file already exists
rb	Read binary
wb	Write binary
r+	Read and write

3. Closing a File

Always close a file after use to free system resources.

```
f = open("example.txt", "r")
print(f.read())
f.close()
```

4. Reading Files

4.1 Read Entire File

```
f = open("example.txt", "r")
content = f.read()
print(content)
f.close()
```

4.2 Read a Specific Number of Characters

```
f = open("example.txt", "r")
print(f.read(10))  # Reads first 10 characters
f.close()
```

4.3 Read Line by Line

```
f = open("example.txt", "r")
print(f.readline())
print(f.readline())
f.close()
```

4.4 Read All Lines as a List

```
f = open("example.txt", "r")
lines = f.readlines()
print(lines)
f.close()
```

4.5 Using a Loop

```
f = open("example.txt", "r")
for line in f:
    print(line)
f.close()
```

5. Writing to Files

5.1 Write Mode (w)

```
f = open("write.txt", "w")
f.write("Hello, Python File Handling!\n")
f.write("This is a new file.")
f.close()
```

6. Appending to Files

Append Mode (a)

Adds content to the end of the file.

```
f = open("write.txt", "a")
f.write("\nThis line is appended.")
f.close()
```

7. Creating Files

Create Mode (x)

```
f = open("newfile.txt", "x")
f.write("New file created successfully.")
f.close()
```

8. Using `with` Statement (Recommended)

The `with` statement automatically closes the file.

```
with open("example.txt", "r") as f:
    data = f.read()
    print(data)
```

Writing with `with`:

```
with open("data.txt", "w") as f:
    f.write("Using with statement")
```

9. File Methods

Method	Description
read()	Reads entire file
readline()	Reads one line
readlines()	Reads all lines into a list
write()	Writes a string to file
writelines()	Writes list of strings
seek()	Changes file cursor position
tell()	Returns current cursor position

Example:

```
with open("example.txt", "r") as f:
    print(f.tell())
    f.seek(5)
    print(f.read())
```

10. Handling Binary Files

Used for images, audio, videos, etc.

```
with open("image.jpg", "rb") as f:
    data = f.read()
```

Writing binary data:

```
with open("copy.jpg", "wb") as f:
    f.write(data)
```

11. File Handling with Exception Handling

```
try:
    f = open("sample.txt", "r")
    print(f.read())
except FileNotFoundError:
    print("File not found")
finally:
    f.close()
```

12. Deleting Files

Using the os module:

```
import os

if os.path.exists("delete.txt"):
    os.remove("delete.txt")
else:
    print("File does not exist")
```

14. Text File Handling (.txt)

Text files store data in human-readable form.

Writing to a Text File

```
with open("sample.txt", "w") as f:
    f.write("Hello World
")
    f.write("This is a text file")
```

Reading from a Text File

```
with open("sample.txt", "r") as f:
    print(f.read())
```

15. JSON File Handling (.json)

JSON (JavaScript Object Notation) is used to store structured data.

Import JSON Module

```
import json
```

Writing JSON to a File

```
data = {
    "name": "Alice",
    "age": 22,
    "skills": ["Python", "Data Analysis"]
}

with open("data.json", "w") as f:
    json.dump(data, f, indent=4)
```

Reading JSON from a File

```
with open("data.json", "r") as f:
    data = json.load(f)
    print(data)
```

Convert JSON String

```
json_string = json.dumps(data)
python_obj = json.loads(json_string)
```

16. CSV File Handling (.csv)

CSV (Comma Separated Values) files store tabular data.

Import CSV Module

```
import csv
```

Writing to a CSV File

```
with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Age", "Grade"])
    writer.writerow(["John", 20, "A"])
    writer.writerow(["Emma", 21, "B"])
```

Reading from a CSV File

```
with open("students.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Using DictReader and DictWriter

```
with open("students.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["Name"], row["Age"])

with open("students2.csv", "w", newline="") as f:
    fieldnames = ["Name", "Age"]
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerow({"Name": "Liam", "Age": 23})
```

17. Comparison Summary

File Type	Module Used	Data Format
Text	Built-in	Plain text
JSON	json	Key–Value (Structured)
CSV	csv	Tabular

Thank You!